

IF25-40401 · DEEP LEARNING · MATERI PENDAMPING PERTEMUAN 04

STRUKTUR PROYEK PROFESIONAL DEEP LEARNING: COMPUTER VISION

Best Practices: Arsitektur Modular, Data Pipeline, Engine, Checkpointing, & Konfigurasi

Tim Dosen Deep Learning · Teknik Informatika · Institut Teknologi Sumatera (ITERA)

Setelah mempelajari materi pendamping ini, mahasiswa diharapkan mampu:

- 1 **Menjelaskan limitasi notebook monolitik** dan urgensi modularisasi berbasis berkas `.py` pada proyek Deep Learning.
- 2 **Merancang tata letak repositori standar industri** yang memisahkan tanggung jawab data, model, training engine, konfigurasi, dan utilitas.
- 3 **Mengimplementasikan pipeline data yang modular**, memisahkan logika pembacaan (*Dataset*), augmentasi (*Transforms*), dan pemuatan batch (*DataLoader*).
- 4 **Menerapkan best practices rekayasa**: *stateful checkpointing*, reproduksibilitas eksperimen (*seeding*), *Automatic Mixed Precision* (AMP), dan logging terstruktur.
- 5 **Mentranslasikan eksperimen eksploratif (Colab/Jupyter)** menjadi repositori terstruktur yang siap dijalankan secara mandiri (*headless execution* via CLI).

1. Fondasi & Tata Letak Direktori

- **Bagian 01 - Urgensi Modularitas:** Limitasi notebook monolitik, bahaya *hidden state*, & prinsip *Separation of Concerns* (SoC).
- **Bagian 02 - Anatomi Standar:** Struktur direktori PyTorch (`configs`, `src`, `tests`) & aturan ketat `.gitignore`.

2. Komponen Inti & Rekayasa Sistem

- **Bagian 03 - Dekomposisi Modul:** Pipeline data (`Dataset`, `Transforms`, `DataLoader`), model factory, trainer engine, & config YAML.
- **Bagian 04 - Best Practices:** Stateful checkpointing, determinisme seed, akselerasi AMP, & tracking eksperimen.
- **Bagian 05 - Alur Praktikum:** Inferensi `predict.py`, ekspor ONNX, & checklist profesional.

BAGIAN 01 · LATAR BELAKANG

Mengapa Butuh Struktur Modular?

Transisi dari Exploratory Notebook ke Robust Production Codebase

Masalah Monolithic Notebook

- **Hidden State:** Sel dieksekusi acak; urutan memori berbeda dari urutan tampilan sel.
- **Git Merge Hell:** Berkas `.ipynb` berformat JSON besar; gambar plot & tensor base64 memicu konflik git yang fatal.
- **Sulit Diuji:** Fungsi di dalam sel tidak dapat di-import oleh automated testing (`pytest`).

Kendala Skalabilitas & Remote GPU

- **Headless Server:** Server GPU kampus/cloud (SSH, `tmux`, SLURM) butuh eksekusi CLI tanpa browser.
- **Automasi Hyperparameter:** Menjalankan puluhan variasi eksperimen (*sweep*) butuh argumen CLI terotomasi.
- **Deployment Terputus:** Backend API (FastAPI) tidak dapat memanggil sel notebook secara langsung.

Notebook vs Berkas .py

Menempatkan Alat Sesuai Peruntukan

Karakteristik	Jupyter Notebook (.ipynb)	Modular Scripts (.py)
Fase Utama	Eksplorasi awal, riset visual, EDA	Pengembangan model, training skala penuh
Struktur Kode	Monolitik, linear, berorientasi sel	Terdekomposisi (modul, paket, fungsi)
Version Control	Rawan konflik (format JSON metadata)	Sangat ramah Git (diff baris per baris)
Eksekusi	Interaktif via web browser	Headless via CLI (<code>python train.py</code>)
Konfigurasi	Variabel di-hardcode di dalam sel	Deklaratif (<code>yaml</code> , <code>argparse</code>)
Testing & CI/CD	Sangat sulit diautomasi	Mudah dengan <code>pytest</code> / CI Actions
Deployment	Tidak cocok untuk produksi	Siap di-package, Docker, atau ONNX

Prinsip Kerja Modern

Gunakan **Notebook** hanya untuk *Exploratory Data Analysis* (EDA) dan visualisasi hasil evaluasi post-hoc. Seluruh pipeline data, definisi arsitektur, dan loop pelatihan harus berada dalam berkas **.py modular**.

1. Separation of Concerns

Pemisahan Tanggung Jawab

Logika membaca citra tidak boleh tahu arsitektur CNN; arsitektur CNN tidak boleh tahu loop optimasi; loop optimasi tidak boleh tahu detail penyimpanan log.

2. Single Responsibility

Satu Berkas, Satu Tugas

Setiap berkas `.py` memegang tepat satu tanggung jawab: modul model hanya mendefinisikan layer, modul dataset hanya menghasilkan tensor citra.

3. Explicit Config

Bebas Magic Numbers

Tidak ada angka acak (*magic numbers*) seperti `lr=0.001` di dalam logika model. Seluruh konfigurasi diatur deklaratif di luar kode.

BAGIAN 02 · ARSITEKTUR DIREKTORI

Anatomi Repositori Standar

Struktur Folder Berstandar Industri untuk Computer Vision (PyTorch)

```
cv_project/  
|-- configs/  
|   |-- base.yaml  
|   '-- exp_lenet.yaml  
|-- data/  
|   |-- raw/  
|   '-- processed/  
|-- src/  
|   |-- datasets/  
|   |-- models/  
|   |-- engine/  
|   '-- utils/  
|-- notebooks/  
|-- tests/  
|-- train.py  
|-- predict.py  
|-- evaluate.py  
'-- requirements.txt
```

Keunggulan Tata Letak Ini

- **Modul `src/` Portabel:** Dapat diinstal sebagai package lokal (`pip install -e .`) atau di-import langsung oleh script lain.
- **Entry Point Terpisah:** Titik masuk jelas: `train.py` untuk melatih, `predict.py` untuk inferensi gambar baru, `evaluate.py` untuk uji metrik.
- **Separasi Eksperimen:** Variasi model diuji cukup dengan mengganti berkas konfigurasi di `configs/`, tanpa mengubah satu baris pun kode logika.

Peran Komponen Utama Repositori

Deskripsi Tanggung Jawab Masing-Masing Modul

- `configs/` File konfigurasi (YAML). Menentukan arsitektur, parameter optimasi, scheduler, batch size, augmentasi, dan path data.
- `src/datasets/` Membaca citra, mengonstruksi label numerik, menerapkan augmentasi khusus train vs test, dan membungkus `DataLoader`.
- `src/models/` Berisi arsitektur CNN (LeNet, Custom CNN) dan *factory* `build_model(config)` untuk instansiasi dinamis.
- `src/engine/` Mengelola loop forward-backward pass, kalkulasi loss, optimasi, *mixed precision*, dan validasi.
- `src/utils/` Helper independen: seed deterministik, penyimpanan checkpoint, metrik akurasi, dan setup logger.
- `train.py` Script CLI utama untuk memuat config, menyiapkan data & model, lalu memanggil trainer engine.

Aturan Emas Version Control (.gitignore)

Menjaga Repositori Tetap Ringan, Bersih, dan Aman

```
# .gitignore untuk Proyek Deep Learning

# Dataset & arsip citra mentah
data/
*.tar.gz
*.zip

# Checkpoint model & bobot biner
checkpoints/
*.pth
*.pt
*.onnx

# Log training & outputs
outputs/
runs/
wandb/
*.log

# Cache & virtual environment
__pycache__/*
*.py[cod]
.venv/
.DS_Store
```

Bahaya Mengabaikan .gitignore

- **Repo Membengkak:** Bobot biner (.pth) ratusan MB merusak riwayat Git dan ditolak GitHub (>100MB).
- **Kebocoran Data:** Dataset berhak cipta atau data sensitif tidak boleh ter-commit publik.

Solusi Distribusi Bobot

Bagikan bobot model terlatih via *GitHub Releases*, *Hugging Face Hub*, atau *Google Drive* menggunakan script download

BAGIAN 03 · DEKOMPOSISI MODUL

Membedah Komponen Inti

Data Pipeline, Arsitektur Model, Trainer Engine, & Konfigurasi

1. Pipeline Data: Pembagian 3 Peran Terpisah

Memisahkan Akses Sampel,

Transformasi, dan Batching

A. Dataset Class

Akses Indeks Sampel

Turunan Dataset. Bertugas:

- Total data (`__len__`)
- Ambil citra & label ke- i (`__getitem__`)

B. Transforms

Pengolahan Spasial

Mengubah citra menjadi tensor:

- Resize & Center Crop
- Augmentasi (flip, rotasi)
- Normalisasi (μ, σ)

C. DataLoader

Batch & Multiprocess

Membungkus batch (B, C, H, W):

- Pengacakan (`shuffle=True`)
- Multi-worker CPU loading
- Pinned memory DMA GPU

Prinsip Utama

Jangan mencampur logika pembacaan berkas (I/O) dengan logika augmentasi citra. Memisahkan keduanya membuat modul `transforms.py` dapat diubah kapan saja tanpa merusak dataset loader.

1. Pipeline Data: Train vs Val Transforms

Pembedaan Ketat Data Latih & Uji

```
# src/datasets/transforms.py
import torchvision.transforms as T

def get_train_transforms(img_size=32):
    return T.Compose([
        T.RandomCrop(img_size, padding=4),
        T.RandomHorizontalFlip(p=0.5),
        T.ToTensor(),
        T.Normalize((0.4914, 0.4822, 0.4465),
                    (0.2023, 0.1994, 0.2010))
    ])

def get_val_transforms(img_size=32):
    return T.Compose([
        T.Resize((img_size, img_size)),
        T.ToTensor(),
        T.Normalize((0.4914, 0.4822, 0.4465),
                    (0.2023, 0.1994, 0.2010))
    ])
```

Anti-Pattern Berbahaya

Menerapkan augmentasi acak (seperti crop atau flip) pada data validasi atau tes!

Aturan Transformasi

- **Data Latih:** Stokastik (acak) untuk mencegah overfitting dan memperkaya sudut pandang.
- **Data Validasi/Uji:** Wajib bersifat **deterministik** agar evaluasi metrik konsisten dan adil.

1. Pipeline Data: Optimasi DataLoader GPU

Mencegah Bottleneck I/O CPU

```
# src/datasets/dataloader.py
from torch.utils.data import DataLoader

def build_dataloaders(train_set, val_set, batch_size=64, num_workers=4):
    train_loader = DataLoader(
        train_set, batch_size=batch_size,
        shuffle=True,          # Wajib acak urutan per epoch
        num_workers=num_workers, # Background worker I/O paralel CPU
        pin_memory=True,       # Kunci di RAM untuk transfer DMA cepat ke GPU
        drop_last=True         # Buang sisa batch ganjil agar BatchNorm stabil
    )
    val_loader = DataLoader(
        val_set, batch_size=batch_size,
        shuffle=False,          # Evaluasi tidak perlu acak urutan
        num_workers=num_workers,
        pin_memory=True,
        drop_last=False         # Evaluasi seluruh sampel tanpa sisa
    )
    return train_loader, val_loader
```

2. Arsitektur Model & Factory Pattern

Instansiasi Fleksibel via Registri

```
# src/models/__init__.py
from .lenet import LeNet5
from .custom_cnn import CustomCNN

MODEL_REGISTRY = {
    "lenet5": LeNet5,
    "custom_cnn": CustomCNN,
}

def build_model(config):
    name = config["model"]["name"].lower()
    if name not in MODEL_REGISTRY:
        raise ValueError(f"Model {name} tidak ada!")

    model_cls = MODEL_REGISTRY[name]
    return model_cls(
        num_classes=config["model"]["num_classes"],
        in_channels=config["model"]["in_channels"]
    )
```

Keunggulan Factory Pattern

- **Decoupling:** Script pelatihan (train.py) tidak perlu mengimpor kelas model secara langsung.
- **Eksperimen Bebas Repot:** Mengganti arsitektur dari LeNet ke Custom CNN cukup via file konfigurasi YAML.
- **Validasi Dini:** Registri langsung menolak nama model yang tidak valid sebelum training dimulai.

Tensor Input/Output

```
# tests/test_model.py
import torch
from src.models import build_model

def test_model_forward():
    cfg = {
        "model": {
            "name": "lenet5",
            "num_classes": 10,
            "in_channels": 3
        }
    }
    model = build_model(cfg)

    # Dummy tensor: Batch=2, C=3, H=32, W=32
    x = torch.randn(2, 3, 32, 32)
    out = model(x)

    # Pastikan shape output tepat: (Batch, Classes)
    assert out.shape == (2, 10), f"Error: {out.shape}"
    print("Sanity check lolos: dimensi tensor valid!")
```

Mengapa Wajib Dilakukan?

- Kesalahan tersering pada CNN adalah salah menghitung dimensi *flattening* sebelum FC layer:

$$\text{In FC} = C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$$

- Dummy test hanya butuh 1 detik, mencegah crash setelah berjam-jam loading dataset.

3. Training Engine: Dekomposisi Fungsi

Memisahkan Loop Pelatihan ke `src/engine/`

```
# src/engine/trainer.py
import torch

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train() # Aktifkan mode latih (BatchNorm update, Dropout aktif)
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)

    return running_loss / total, 100.0 * correct / total
```

3. Training Engine: Evaluasi Independen

Evaluasi Bebas dari Graf Autograd

```
# src/engine/evaluator.py
import torch

@torch.no_grad() # Nonaktifkan autograd untuk hemat memori VRAM GPU
def evaluate(model, loader, criterion, device):
    model.eval() # Mode evaluasi (BatchNorm freeze statistik, Dropout mati)
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)

    return running_loss / total, 100.0 * correct / total
```

4. Manajemen Konfigurasi

Pemisahan Parameter via YAML

```
# configs/experiments/lenet_cifar10.yaml
experiment_name: "lenet5_cifar10_baseline"
seed: 42
```

```
dataset:
  name: "CIFAR10"
  root: "./data"
  img_size: 32
  num_classes: 10
```

```
model:
  name: "lenet5"
  in_channels: 3
  num_classes: 10
```

```
training:
  epochs: 30
  batch_size: 128
  num_workers: 4
  lr: 0.001
  weight_decay: 0.0001
  patience: 7
```

Tingkatan Konfigurasi Proyek

- 1 Level 1:** argparse **Saja** Bawaan Python, bagus untuk script cepat namun repot jika parameter banyak.
- 2 Level 2: YAML + CLI Override** (*Best Practice Disarankan*) File YAML menyimpan parameter default, CLI mengizinkan override cepat.
- 3 Level 3: Hydra / OmegaConf** Standar lab riset enterprise dengan fitur hierarki konfigurasi modular.

5. Utilitas Esensial: Logger & Metrik

Pencatatan Riwayat Latih Terstruktur

```
# src/utlis/logger.py
import logging, sys

def setup_logger(log_file="outputs/train.log"):
    logger = logging.getLogger("CV_Project")
    logger.setLevel(logging.INFO)
    logger.handlers.clear()

    fmt = logging.Formatter(
        "[%asctime)s] [%levelname)s] %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S"
    )

    # Stream ke terminal konsol
    ch = logging.StreamHandler(sys.stdout)
    ch.setFormatter(fmt)
    logger.addHandler(ch)

    # Tulis permanen ke berkas log disk
    fh = logging.FileHandler(log_file)
    fh.setFormatter(fmt)
    logger.addHandler(fh)

    return logger
```

Mengapa Logging Sangat Krusial?

- **Terminal Terputus:** Saat koneksi SSH terputus, teks terminal hilang. File log menyimpan riwayat permanen.
- **Audit Eksperimen:** Setiap epoch mencatat timestamp, loss, akurasi, dan update learning rate.
- **Level Pesan:** Membedakan informasi biasa (INFO), peringatan (WARNING), dan error (ERROR).

BAGIAN 04 · REKAYASA SISTEM

Best Practices Engineering

Reproducibility, Stateful Checkpoint, AMP, & Tracking

Menghasilkan Angka Identik

```
# src/utils/seed.py
import os, random, numpy as np, torch

def seed_everything(seed=42):
    random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)  # Untuk lingkungan multi-GPU

# Konfigurasi cuDNN deterministik
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Kapan Dipanggil?

Fungsi `seed_everything(config["seed"])` wajib dipanggil di baris paling awal eksekusi `train.py` sebelum pembuatan dataset dan inisialisasi bobot jaringan.

```
# src/utils/checkpoint.py
import torch

def save_checkpoint(state, is_best, filepath="
    checkpoints/last.pth"):
    torch.save(state, filepath)
    if is_best:
        torch.save(state, "checkpoints/best.pth")

# Struktur state dict lengkap:
checkpoint = {
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "scheduler_state_dict": scheduler.state_dict(),
    "best_val_acc": best_val_acc,
    "config": config
}
```

Bahaya Stateless Checkpoint

Jika hanya menyimpan bobot model tanpa status optimizer dan scheduler, pelatihan **tidak dapat dilanjutkan (*resume*)** saat server mati! Momentum optimizer akan ter-reset ke awal.

Strategi Dual Checkpoint

- `last.pth`: Ditimpa per epoch untuk resume training.
- `best.pth`: Disimpan saat val loss / akurasi memecahkan rekor terbaik.

```
# Pola Modern PyTorch AMP (torch.amp)
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

for images, labels in loader:
    images, labels = images.to("cuda"), labels.to("cuda")
    optimizer.zero_grad()

    # Forward pass dengan presisi campuran FP16/BF16 otomatis
    with torch.amp.autocast("cuda", enabled=use_amp):
        outputs = model(images)
        loss = criterion(outputs, labels)

    # Backward pass dengan dynamic loss scaling (mencegah underflow gradien)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Manfaat Nyata AMP pada GPU Modern (T4, RTX, A100)

Menghemat memory GPU (VRAM) hingga $\sim 50\%$, mempercepat komputasi $2 \times - 3 \times$ lipat berkat Tensor Cores, tanpa menurunkan akurasi model akhir.

TensorBoard (Lokal & Mandiri)

- Bawaan pustaka
`torch.utils.tensorboard`.
- Menyimpan event file secara lokal di `outputs/runs/`.
- Menampilkan kurva *loss train vs val* di browser: `tensorboard -logdir outputs/runs`
- Visualisasi matriks konfusi & prediksi model.

Weights & Biases (WandB - Cloud)

- Standar kolaborasi tim industri dan riset.
- Dashboard berbasis cloud untuk seluruh anggota tim.
- Komparasi otomatis puluhan eksperimen.
- Menyimpan grafik performa terpusat.

Gunakan tracker untuk mendokumentasikan bukti eksperimen secara ilmiah dan terpercaya.

BAGIAN 05 · ALUR KERJA & PRAKTIKUM

Inferensi, Ekspor, & Praktikum

Standalone Predictor, Ekspor ONNX, dan Roadmap Migrasi Mahasiswa

```
# predict.py
import argparse, torch
from PIL import Image
from src.models import build_model
from src.datasets.transforms import get_val_transforms

def predict(image_path, checkpoint_path):
    # 1. Muat checkpoint & arsitektur model
    ckpt = torch.load(checkpoint_path, map_location="cpu")
    model = build_model(ckpt["config"])
    model.load_state_dict(ckpt["model_state_dict"])
    model.eval()

    # 2. Preprocessing deterministik
    transform = get_val_transforms(img_size=32)
    img = Image.open(image_path).convert("RGB")
    x = transform(img).unsqueeze(0) # Dimensi batch: (1, 3, 32, 32)

    # 3. Inferensi
    with torch.no_grad():
        probs = torch.softmax(model(x), dim=1)
        conf, pred_id = probs.max(1)

    return pred_id.item(), conf.item()
```

```
# export_onnx.py
import torch
from src.models import build_model

def export_to_onnx(ckpt_path, output_onnx):
    ckpt = torch.load(ckpt_path, map_location="cpu")
    model = build_model(ckpt["config"])
    model.load_state_dict(ckpt["model_state_dict"])
    model.eval()

    dummy_input = torch.randn(1, 3, 32, 32)
    torch.onnx.export(
        model, dummy_input, output_onnx,
        input_names=["input_image"],
        output_names=["class_logits"],
        opset_version=14
    )
    print(f"Model berhasil diekspor ke: {output_onnx}")
```

Mengapa ONNX?

- **Framework Agnostic:** Menghapus ketergantungan runtime Python/PyTorch.
- **Deployment Luas:** Siap dijalankan di C++, mobile (iOS/Android), web browser (ONNX Runtime Web), dan edge (TensorRT).
- **Latensi Rendah:** Eksekusi inferensi hingga $3 \times - 5 \times$ lebih cepat di server produksi.

Fase 1: Dekomposisi Modul

- 1 Ekstrak Dataset & Transforms:**
Pindahkan dataset CIFAR-10 dan normalisasi dari sel notebook ke modul `src/datasets/`.
- 2 Ekstrak Kelas Model:**
Pindahkan kelas `LeNet5` ke `src/models/lenet.py` dan registrasikan di `__init__.py`.
- 3 Buat File Konfigurasi:**
Pindahkan seluruh hiperparameter (`lr`, `batch size`) ke `configs/exp_lenet.yaml`.

Fase 2: Eksekusi & Testing

- 4 Bangun Trainer Engine:**
Pindahkan loop epoch ke `src/engine/trainer.py` dengan dukungan checkpointing.
- 5 Susun Entry Point CLI:**
Tulis `train.py` yang menerima argumen konfigurasi via terminal.
- 6 Jalankan Eksperimen:**

```
python train.py -config  
configs/exp_lenet.yaml
```

Version Control & Konfigurasi

- ✓ Ada `.gitignore` memblokir dataset dan bobot `*.pth`?
- ✓ Bebas dari *magic numbers* (hiperparameter di YAML/CLI)?
- ✓ Fungsi `seed_everything` aktif untuk reproduksibilitas?
- ✓ Dependensi tercantum di `requirements.txt`?

Engineering & Deployment

- ✓ Transform train stokastik terpisah dari transform val deterministik?
- ✓ Checkpoint menyimpan optimizer state & scheduler state?
- ✓ `model.train()` dan `model.eval()` ditempatkan tepat?
- ✓ Tersedia script `predict.py` mandiri untuk uji citra baru?

Patuhi checklist ini untuk memastikan kode Anda berstandar industri dan mudah diaudit.

IF25-40401 · DEEP LEARNING

TERIMA KASIH

Selamat Membangun Proyek Deep Learning yang Modular, Teruji, dan Siap Produksi.

Teknik Informatika — Institut Teknologi Sumatera (ITERA) · 2026